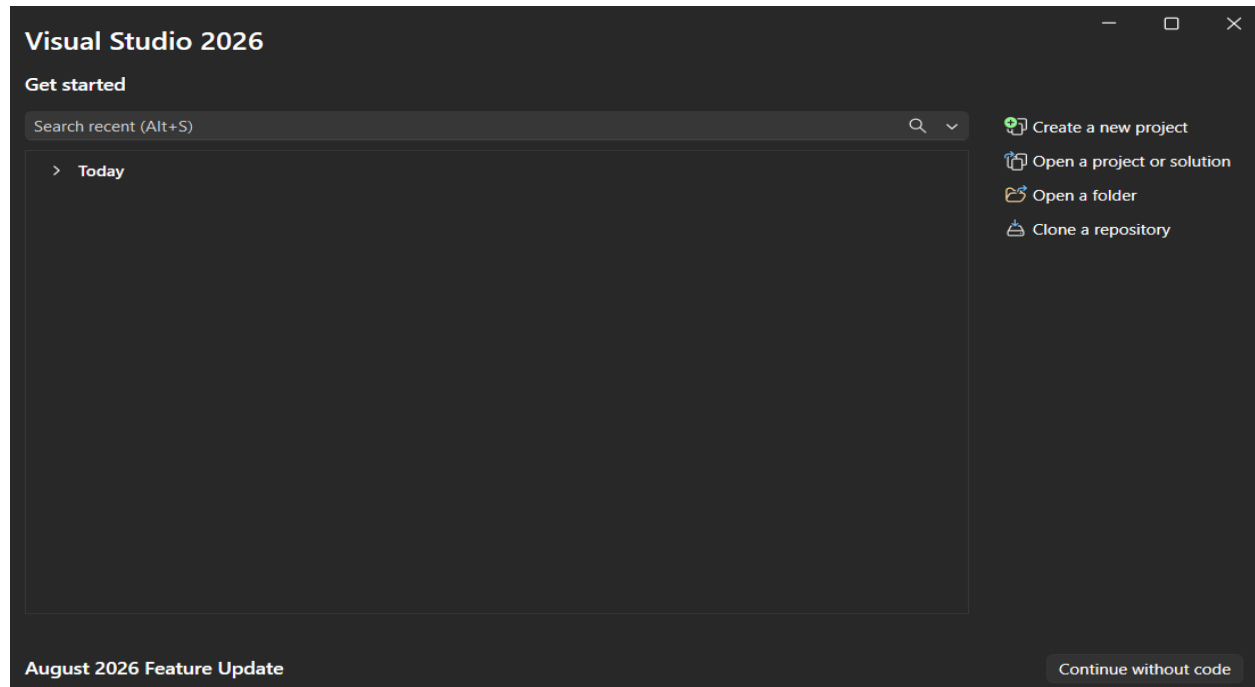


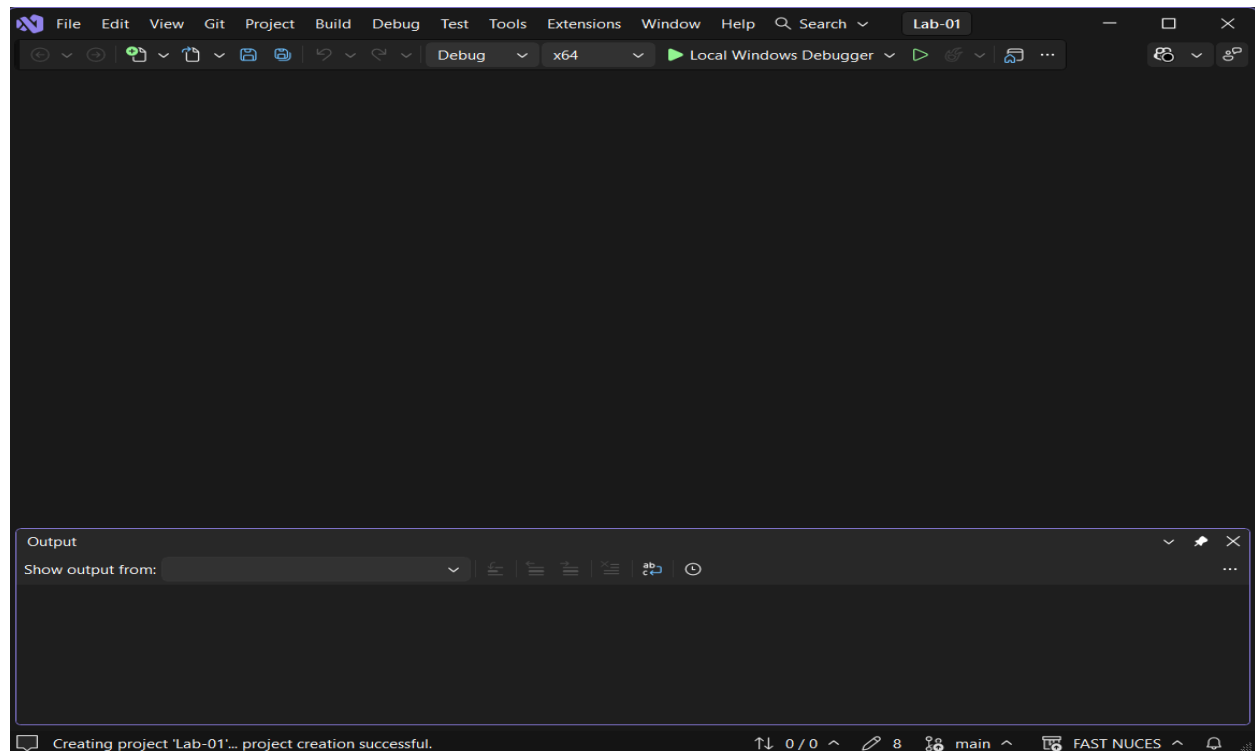
COAL LAB 1

Task#01:

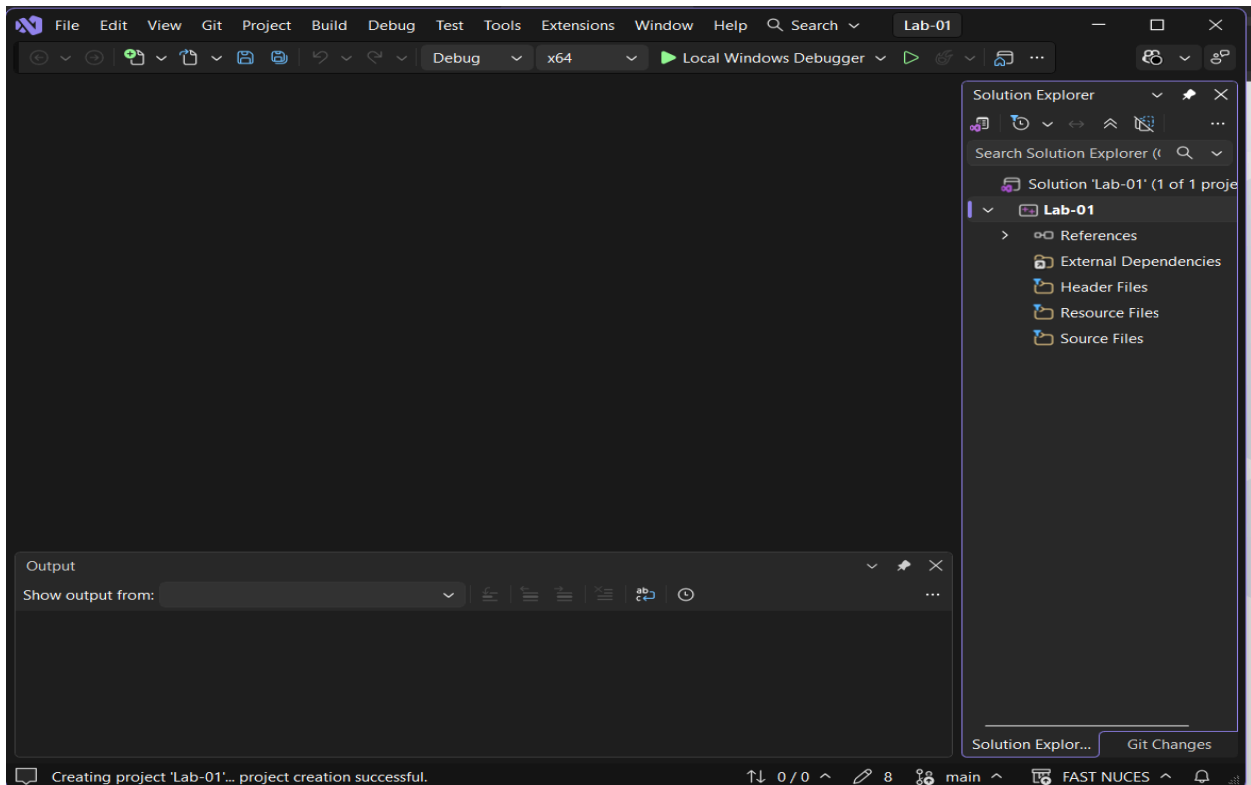
Step 04: Create new project



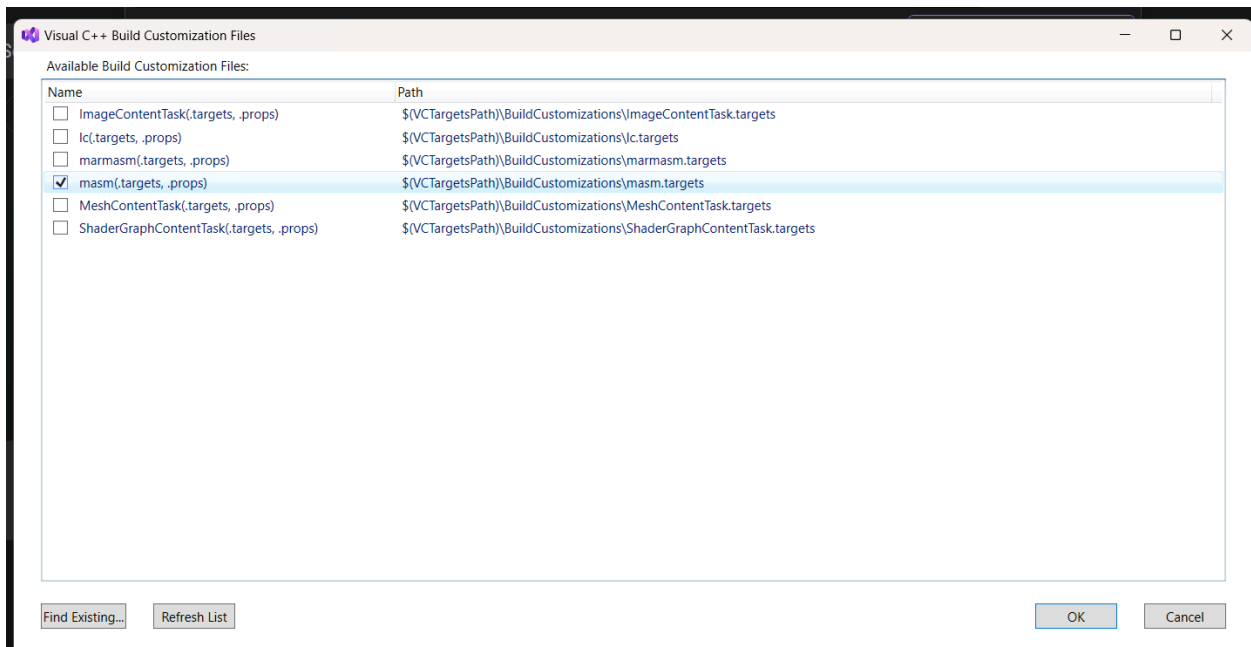
Step 05: Created an empty project (language: C++)



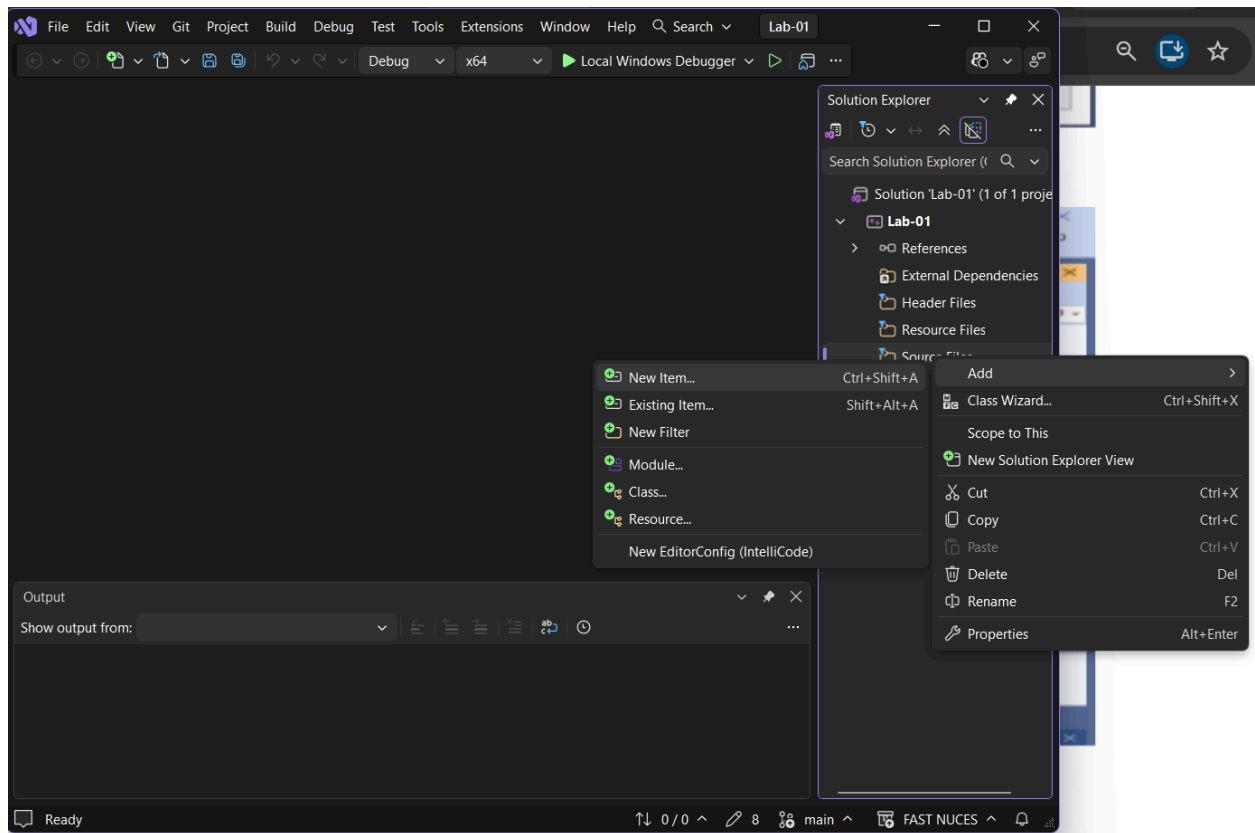
Step 06: Open Solution Explorer. (ctrl + Alt + L)



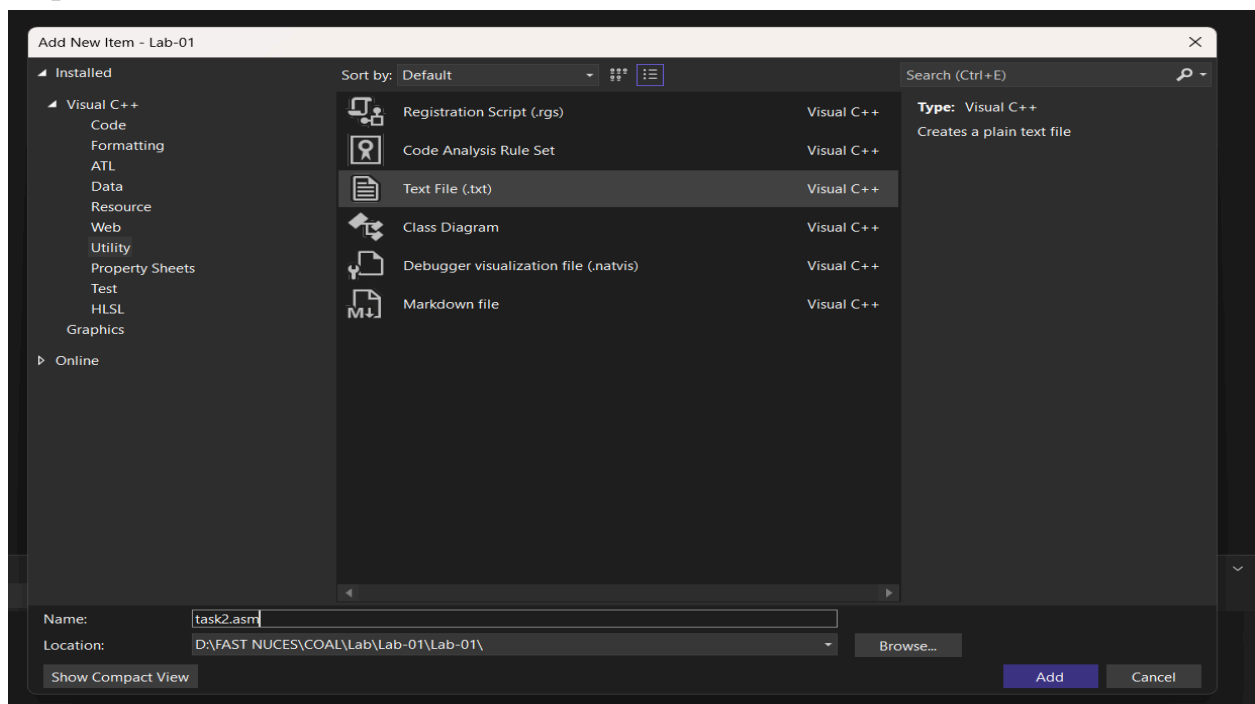
Step 07 & 08: Build customization and tick masm checkbox.



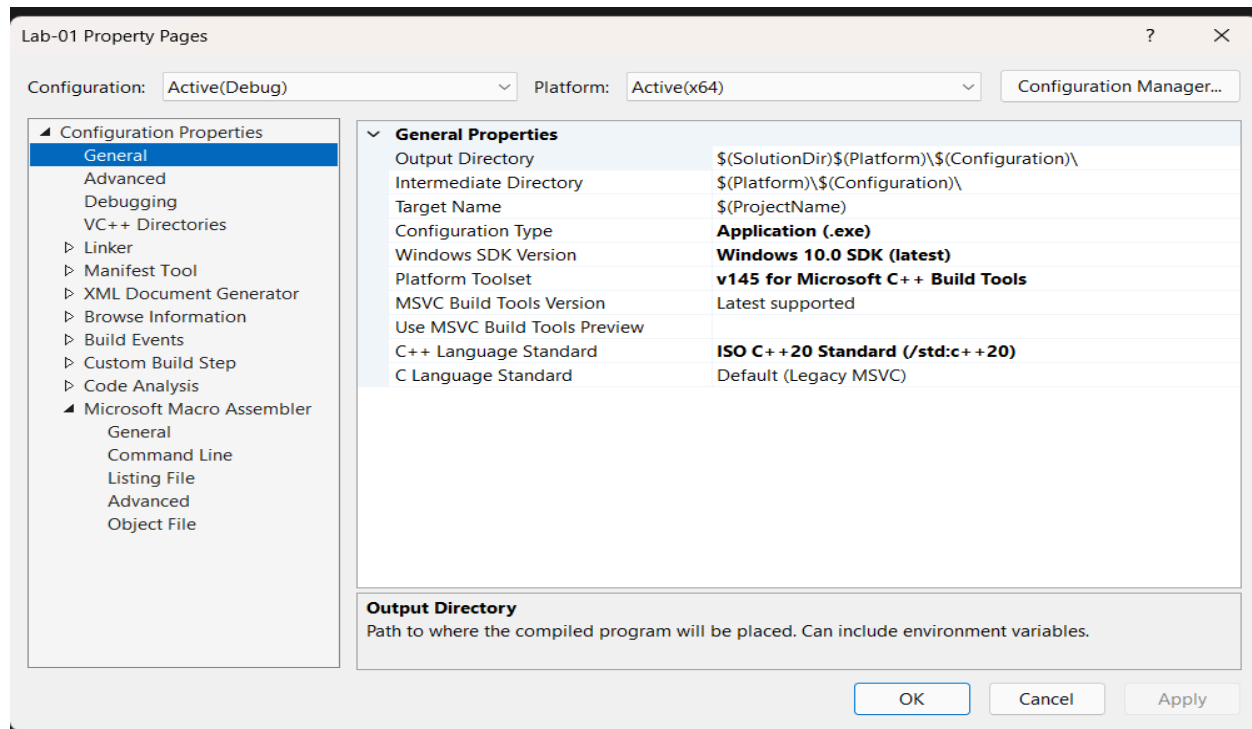
Step 09: Source file > add > new item.



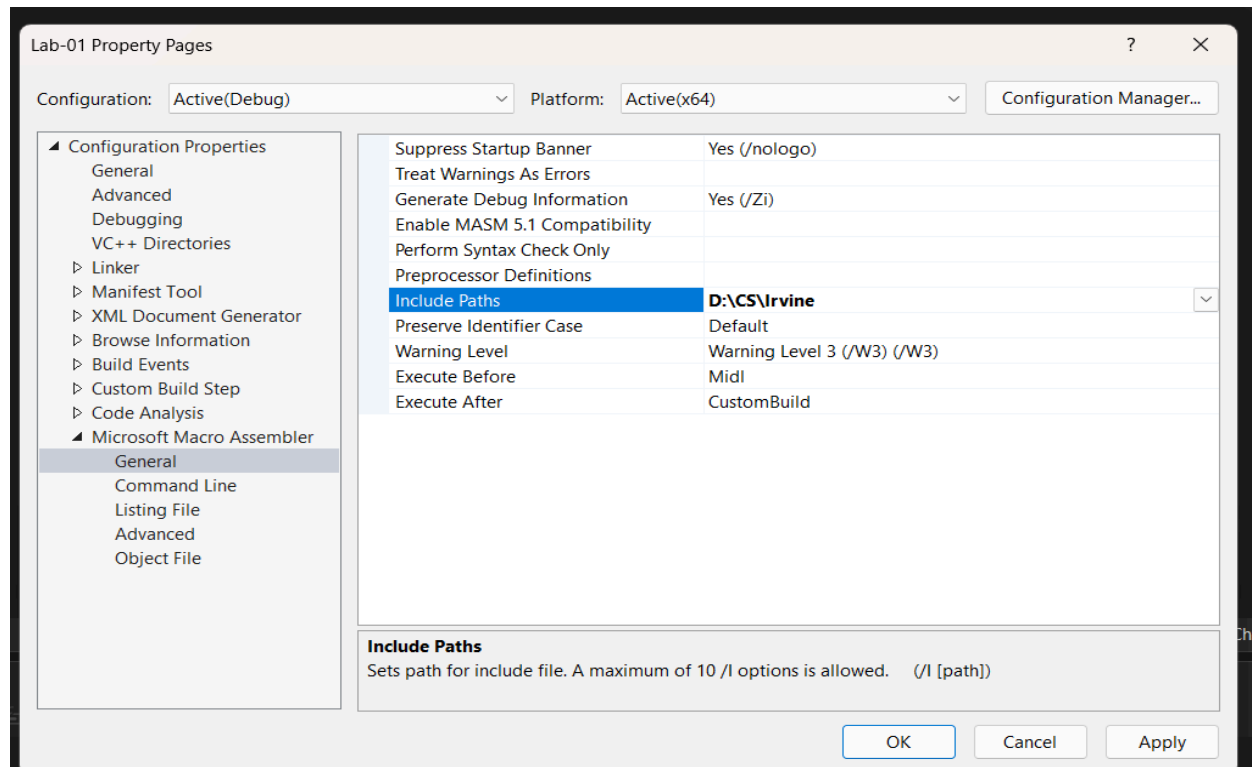
Step 10: Create .asm file.



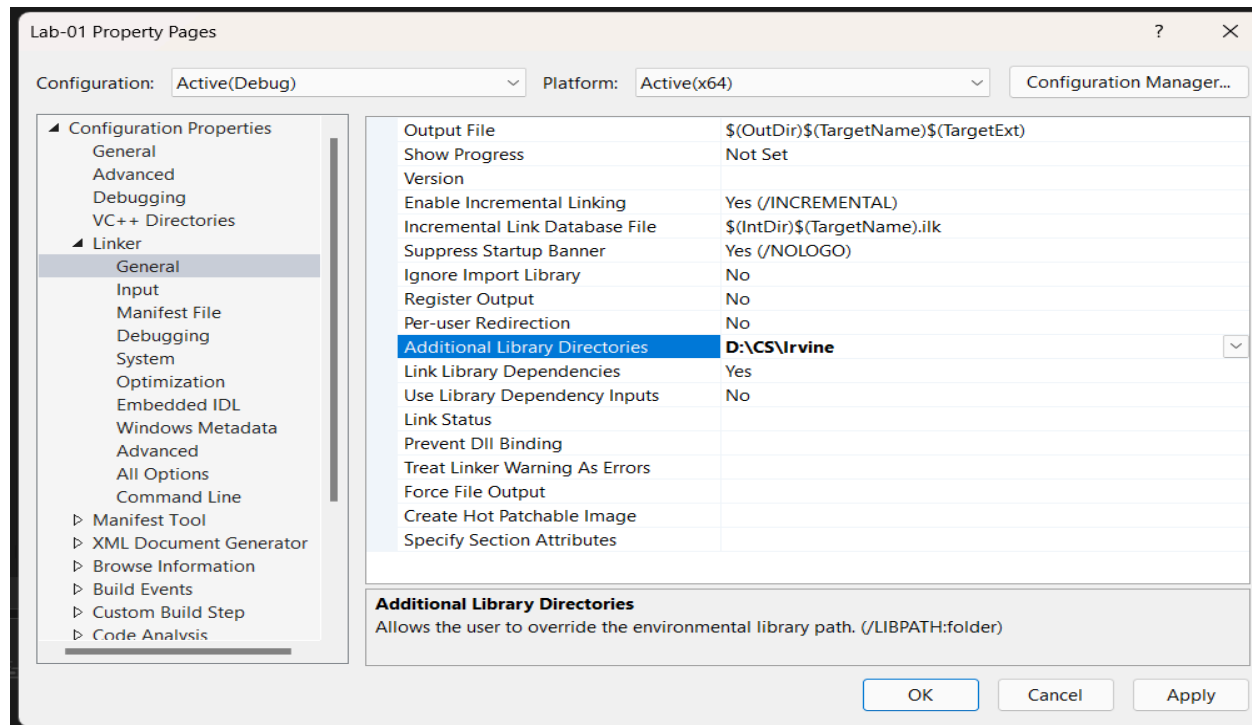
Step 11: Expand Microsoft Macro Assembler.



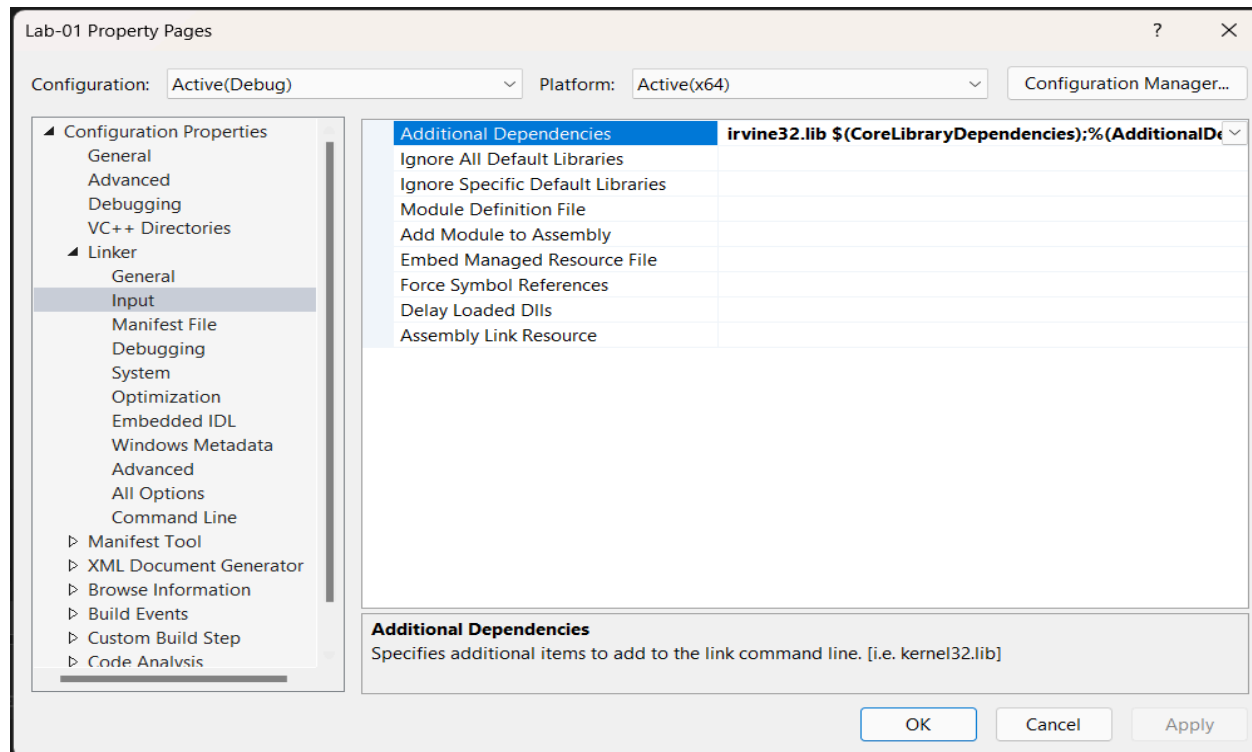
Step 12: Include Irvine path.



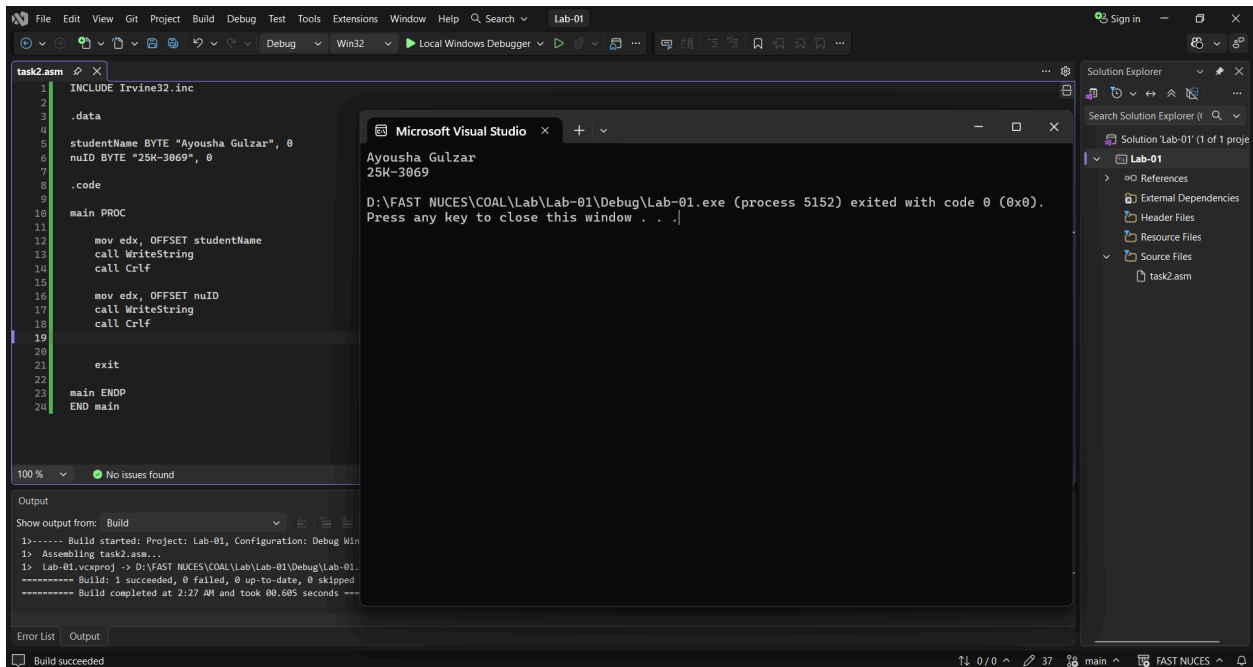
Step 13: Add additional library directories.



Step 14: Add irvine32.lib in input > additional dependencies.



Step 03: Modify the program to print your name and NU-ID on separate lines instead of a single line.



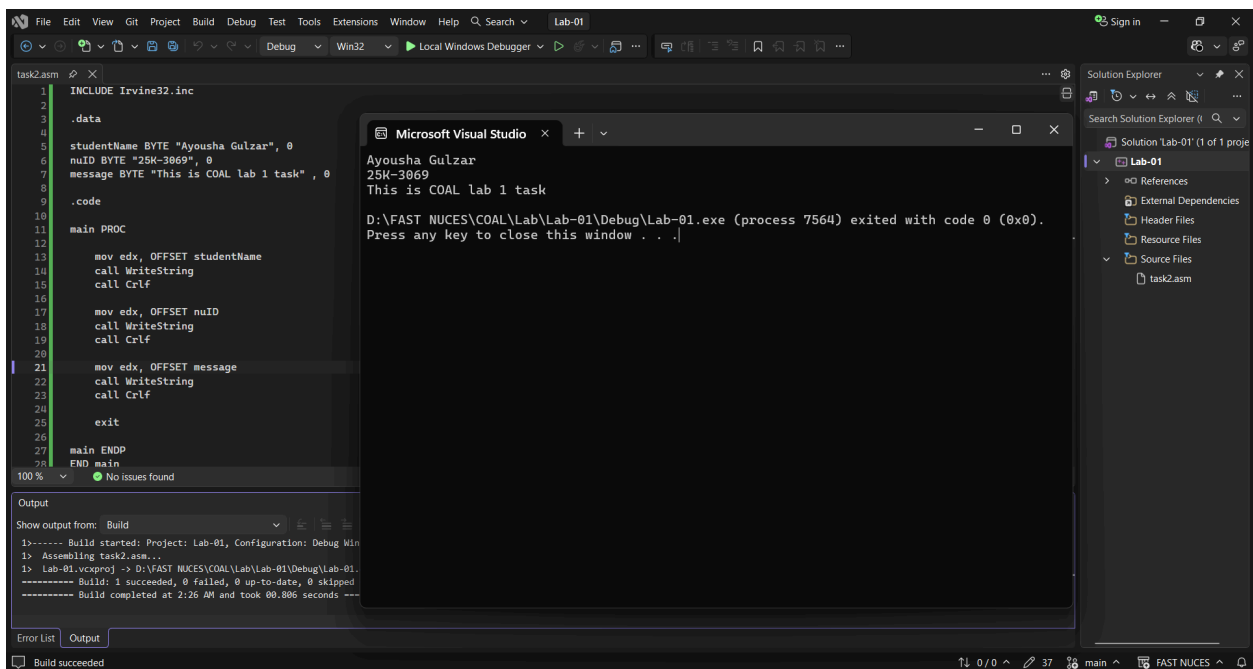
The screenshot shows the Microsoft Visual Studio IDE with the assembly file `task2.asm` open. The code includes `Irvine32.inc` and defines a `.data` section with `studentName` (a string) and `nuID` (a string). The `.code` section contains a `main` procedure that uses `WriteString` to print the name and ID on separate lines, separated by a carriage return and line feed (`CRLF`). The program is built and executed, showing the output in the Output window: "Ayousha Gulzar" followed by "25K-3069". The Solution Explorer on the right shows the project structure for "Lab-01".

```
1 INCLUDE Irvine32.inc
2
3 .data
4
5 studentName BYTE "Ayousha Gulzar", 0
6 nuID BYTE "25K-3069", 0
7
8 .code
9
10 main PROC
11
12     mov edx, OFFSET studentName
13     call WriteString
14     call Crlf
15
16     mov edx, OFFSET nuID
17     call WriteString
18     call Crlf
19
20
21     exit
22
23 main ENDP
24 END main
```

Output:

```
1>----- Build started: Project: Lab-01, Configuration: Debug Win32
1> Assembling task2.asm...
1> Lab-01.vcxproj -> D:\FAST NUCES\COAL\Lab\Lab-01\Debug\Lab-01.exe
----- Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped
----- Build completed at 2:27 AM and took 00.605 seconds -----
```

Step 05: Extend the program to display an additional personalized message of your choice.



The screenshot shows the Microsoft Visual Studio IDE with the modified assembly file `task2.asm`. The code now includes a `message` string in the `.data` section. The `main` procedure is extended to print this message after the name and ID. The program is built and executed, showing the output in the Output window: "Ayousha Gulzar", "25K-3069", and "This is COAL lab 1 task". The Solution Explorer on the right shows the project structure for "Lab-01".

```
1 INCLUDE Irvine32.inc
2
3 .data
4
5 studentName BYTE "Ayousha Gulzar", 0
6 nuID BYTE "25K-3069", 0
7 message BYTE "This is COAL lab 1 task", 0
8
9 .code
10
11 main PROC
12
13     mov edx, OFFSET studentName
14     call WriteString
15     call Crlf
16
17     mov edx, OFFSET nuID
18     call WriteString
19     call Crlf
20
21     mov edx, OFFSET message
22     call WriteString
23     call Crlf
24
25     exit
26
27 main ENDP
28 END main
```

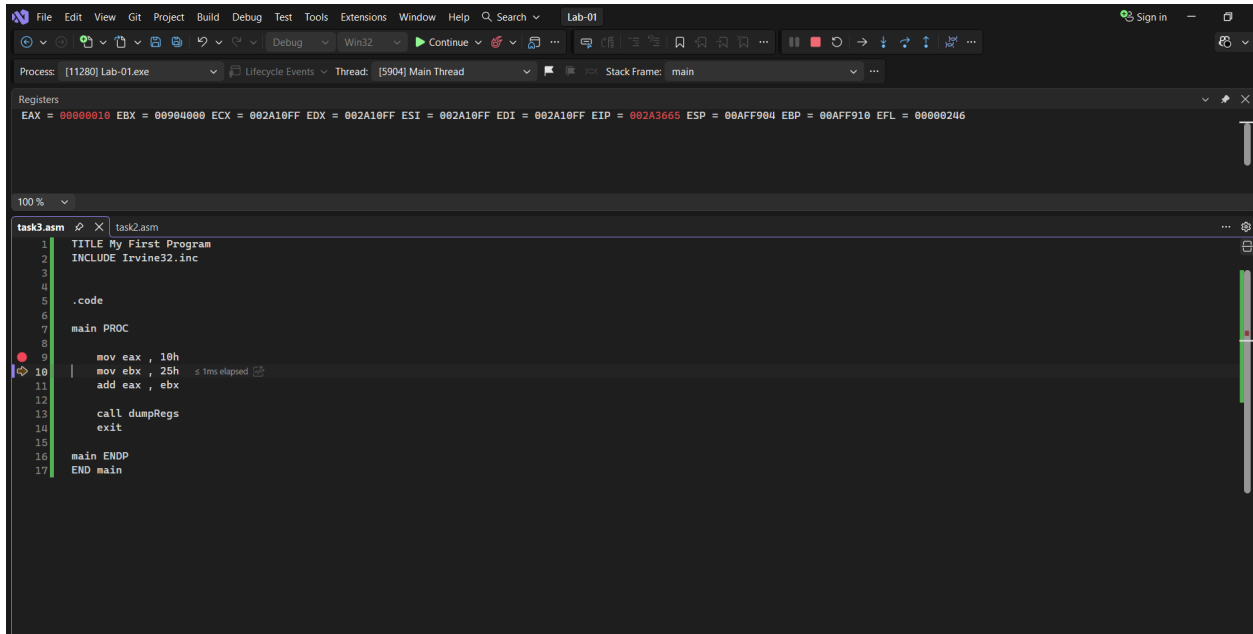
Output:

```
1>----- Build started: Project: Lab-01, Configuration: Debug Win32
1> Assembling task2.asm...
1> Lab-01.vcxproj -> D:\FAST NUCES\COAL\Lab\Lab-01\Debug\Lab-01.exe
----- Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped
----- Build completed at 2:26 AM and took 00.806 seconds -----
```

Task#03:

3.1: After “mov eax, 10h”

- EAX = 00000010h
- EBX = unchanged



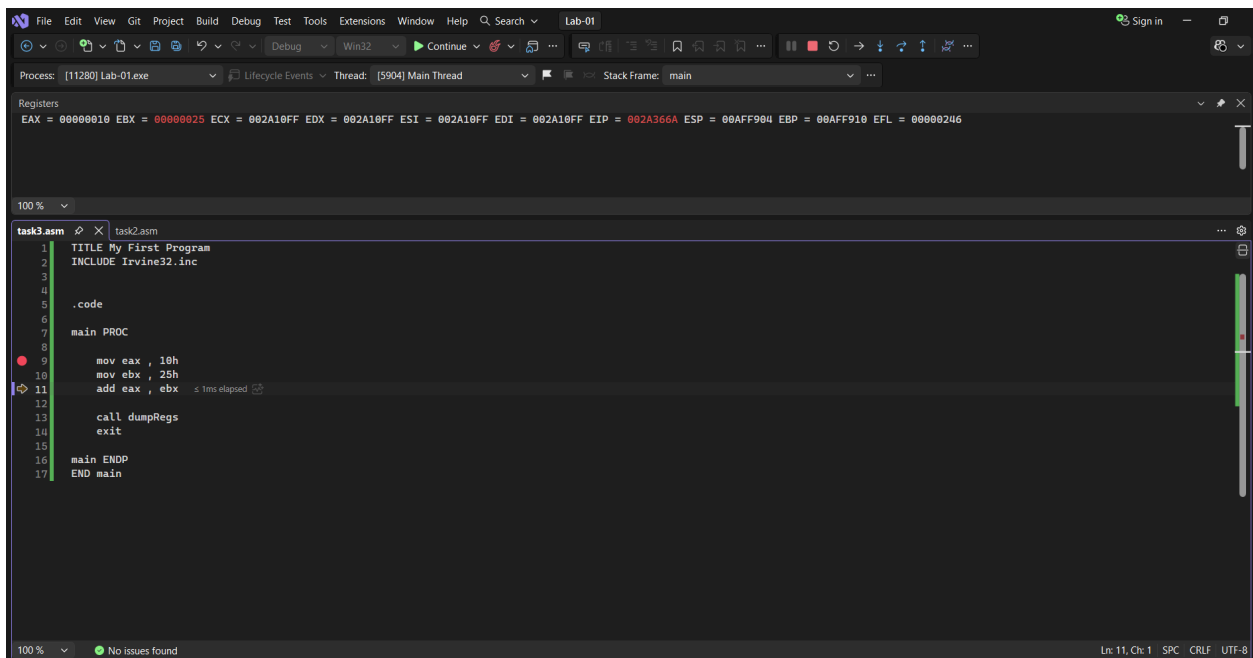
Registers

EAX = 00000010 EBX = 00904000 ECX = 002A10FF EDX = 002A10FF ESI = 002A10FF EDI = 002A10FF EIP = 002A3665 ESP = 00AFF904 EBP = 00AFF910 EFL = 00000246

```
1  TITLE My First Program
2  INCLUDE Irvine32.inc
3
4
5  .code
6
7  main PROC
8
9  mov eax, 10h
10 | mov ebx, 25h
11 | add eax, ebx
12
13 call dumpRegs
14 exit
15
16 main ENDP
17 END main
```

After “mov ebx, 25h”

- EAX = 00000010h
- EBX = 00000025h



Registers

EAX = 00000010 EBX = 00000025 ECX = 002A10FF EDX = 002A10FF ESI = 002A10FF EDI = 002A10FF EIP = 002A366A ESP = 00AFF904 EBP = 00AFF910 EFL = 00000246

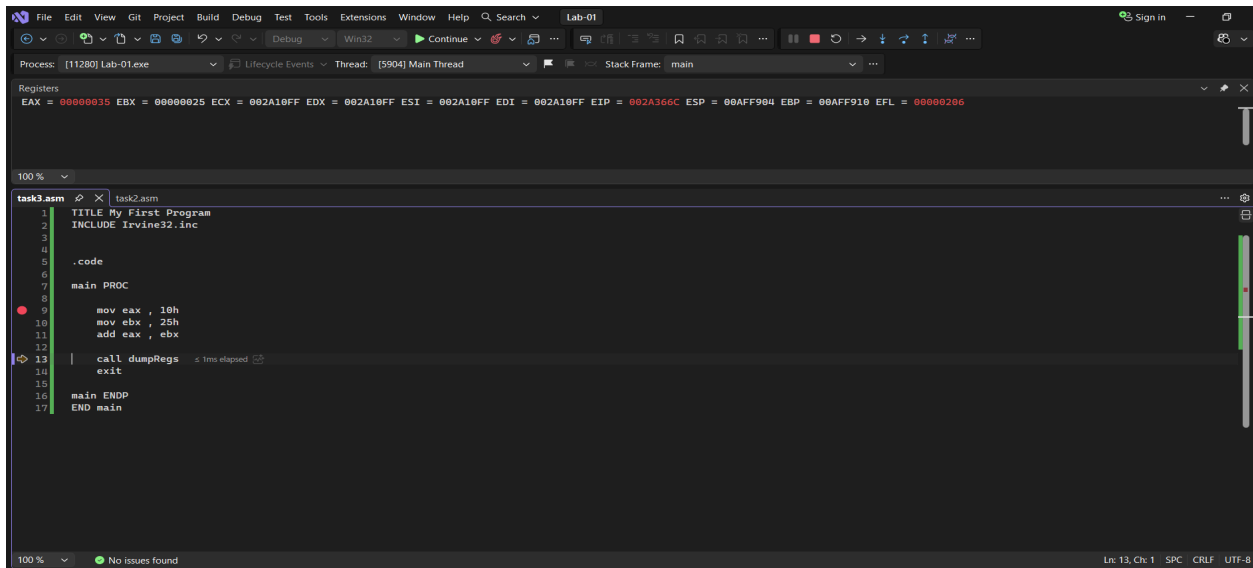
```
1  TITLE My First Program
2  INCLUDE Irvine32.inc
3
4
5  .code
6
7  main PROC
8
9  mov eax, 10h
10 | mov ebx, 25h
11 | add eax, ebx
12
13 call dumpRegs
14 exit
15
16 main ENDP
17 END main
```

100 % No issues found

Ln 11, Ch 1 SPC CRLF UTF-8

After “add eax,ebx”

- EAX = 00000035h
- EBX = 00000025h



The screenshot shows the Visual Studio IDE with the assembly code for task3.asm. The code is as follows:

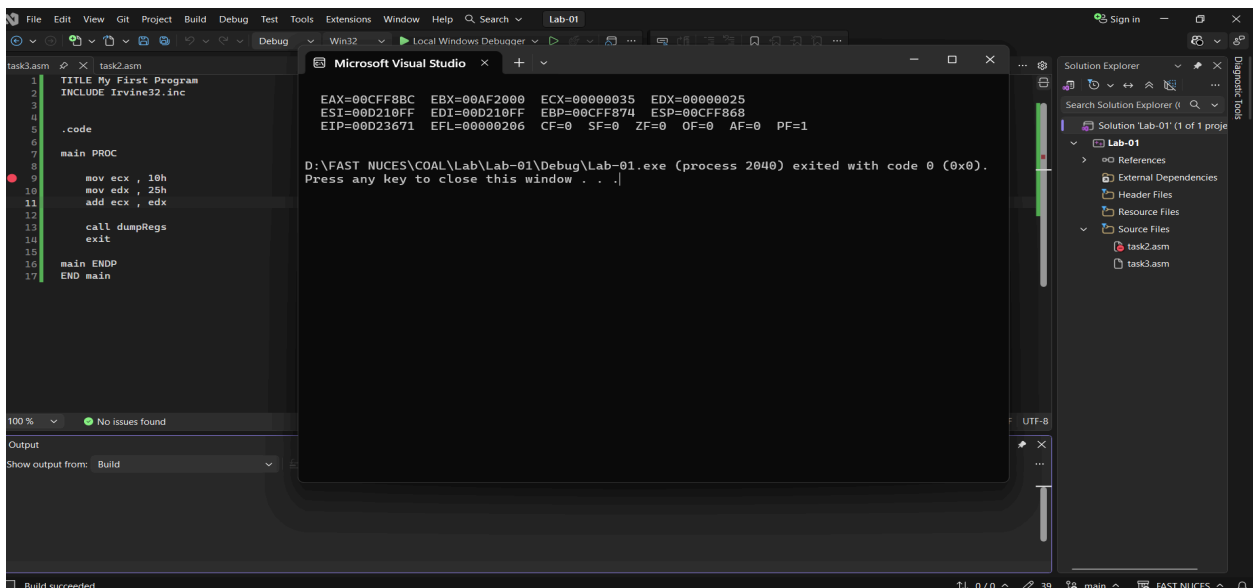
```
1  TITLE My First Program
2  INCLUDE Irvine32.inc
3
4
5  .code
6
7  main PROC
8
9      mov eax, 10h
10     mov ebx, 25h
11     add eax, ebx
12
13     call dumpRegs
14     exit
15
16 main ENDP
17 END main
```

The Register window at the top shows the following values:

Register	Value
EAX	00000035
ECX	00000025
EDX	002A10FF
ESI	002A10FF
EDI	002A10FF
EIP	002A366C
ESP	00AFF904
EBP	00AFF910
EFL	00000206

10h + 25h = 35h, which matches the value obtained in EAX after the ADD instruction.

3.2:



The screenshot shows the Visual Studio IDE with the assembly code for task3.asm. The code is as follows:

```
1  TITLE My First Program
2  INCLUDE Irvine32.inc
3
4
5  .code
6
7  main PROC
8
9      mov ecx, 10h
10     mov edx, 25h
11     add ecx, edx
12
13     call dumpRegs
14     exit
15
16 main ENDP
17 END main
```

The Register window at the top shows the following values:

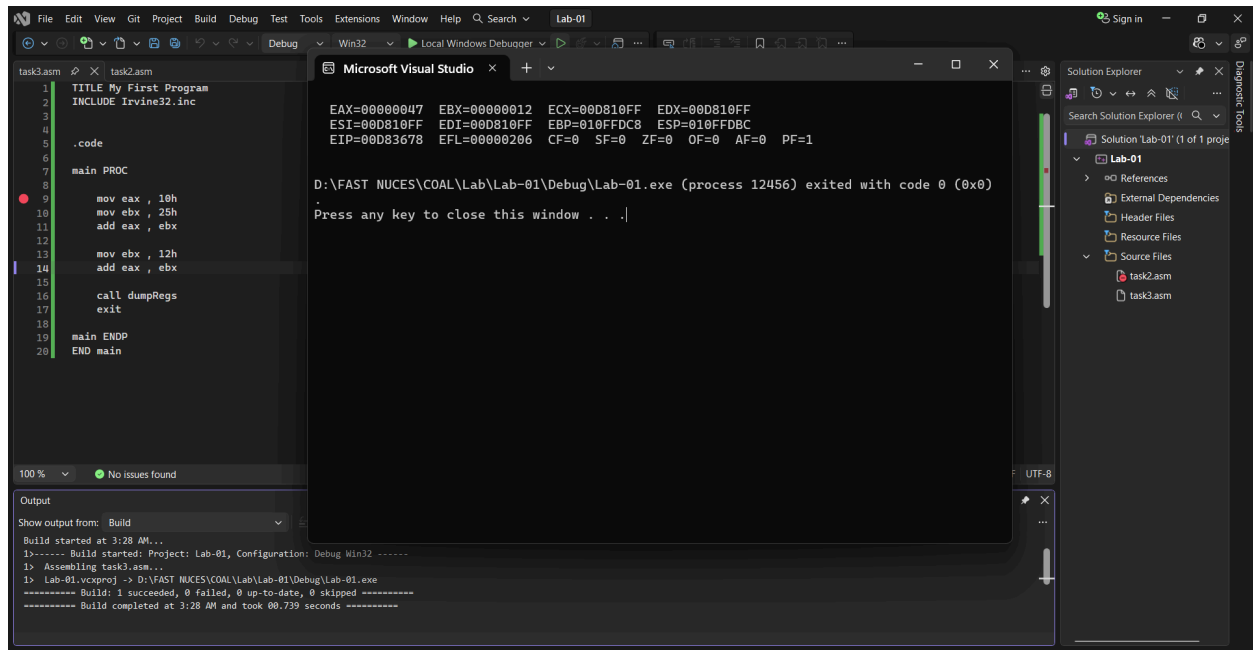
Register	Value
EAX	00CFF8BC
EBX	00AF2000
ECX	00000035
EDX	00000025
ESI	00D218FF
EDI	00D218FF
EBP	00CFF874
ESP	00CFF868
EIP	00D23671
EFL	00000206
CF	0
SF	0
ZF	0
OF	0
AF	0
PF	1

The program behavior does not change because ECX and EDX are also registers which are capable of performing the same operations. Only the registers storing the values are changed; the calculation remains the same.

3.3: I will use EAX and EBX registers with MOV to load the three numbers and ADD to calculate their sum in EAX.

For example, MOV EAX, 10h, MOV EBX, 25h, ADD EAX, EBX, then load the third number into EBX and perform another ADD.

Here is the code:



The screenshot shows the Microsoft Visual Studio IDE. On the left, the 'task3.asm' file is open, displaying assembly code. The code includes Irvine32.inc, defines a main PROC, and contains instructions: mov eax, 10h, mov ebx, 25h, add eax, ebx, mov ebx, 12h, add eax, ebx, call dumpRegs, and exit. The main PROC ends with main ENDP and END main. The bottom status bar shows '100%' zoom and 'No issues found'. On the right, the 'Microsoft Visual Studio' debugger window is open, showing the current state of registers: EAX=00000047, EBX=00000012, ECX=00D810FF, EDX=00D810FF, ESI=00D810FF, EDI=00D810FF, EBP=010FFDC8, ESP=010FFDBC, EIP=00D83678, EFL=00000206, CF=0, SF=0, ZF=0, OF=0, AF=0, PF=1. Below the register dump, it states 'D:\FAST NUCES\COAL\Lab\Lab-01\Debug\Lab-01.exe (process 12456) exited with code 0 (0x0)' and 'Press any key to close this window . . .'. The Solution Explorer on the far right shows the project structure for 'Lab-01', including task2.asm and task3.asm.

3.4: Table (for original code)

Registers	Before ADD	After ADD
EAX	00000010h	00000035h
EBX	00000025h	00000025h

3.5:

DumpRegs helps us check the current state of the registers after the instructions are executed. It displays the values of registers like EAX, EBX, ECX, and EDX, along with the status flags, so we can verify our calculations.

3.6:

MOV is used to store a value into a register. When `mov "eax, 10h"` is executed, the value 10h is stored into EAX, so EAX becomes 00000010h.

Task#04:

Lab 1 gave me a practical understanding of how an Assembly program is developed and executed. I learned that MASM processes the Assembly source code, translates its instructions into machine-level object code, and generates an **.obj file**. The Linker then resolves the external references and combines this object file with required libraries, such as Irvine32.lib, to produce the final **.exe** executable. I also learned that **registers** are used to hold data during execution, **MOV** transfers values and **ADD** performs arithmetic, and Irvine32 procedures can be called to perform predefined tasks. Overall, the lab helped me understand the connection between Assembly code and the final executable program.

Task#05 - Importance of Assembly Language:

Assembly Language is still important because it provides direct control over hardware, including CPU registers and memory. Although languages like Java, C++, and Python simplify programming, Assembly helps achieve better performance and efficiency in hardware-dependent applications. It is widely useful in areas such as embedded systems, operating systems, device drivers, and performance-critical software. Learning Assembly also develops a deeper understanding of computer architecture, instruction execution, memory management, and debugging. This low-level knowledge strengthens programming fundamentals and helps programmers better understand how high-level languages interact with computer hardware.